

ALGORITHMIQUES DE GRAPHS & OPTIMISATION

CH.II : CHEMINS EXTRÉMAUX DES GRAPHS PONDÉRÉS

II1: 2023/2024

Plan du chapitre

- Typologie des problèmes d'optimisation
- Problèmes Dérivés
- Application : Le problème central de l'ordonnancement
(PERT-MPM)

Partie 1:

Typologie des problèmes du plus court chemin « PCC »

(3)

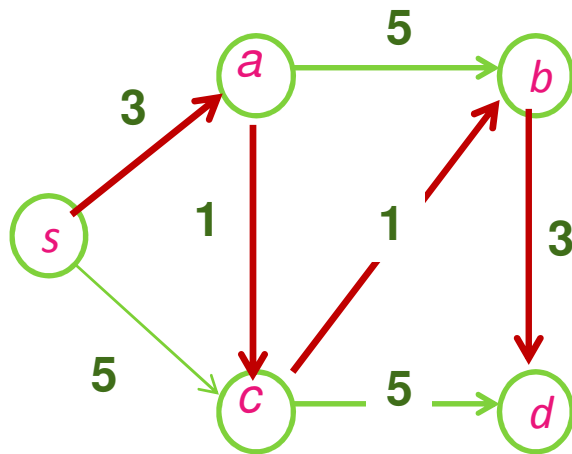
1-1- INTRODUCTION

- On considère un graphe $G = (S, A)$ orienté et valué.
- Le problème de la recherche d'un plus court *chemin dans un graphe* orienté a de très nombreuses applications pratiques, telles :
 - Routages de paquets dans les réseaux, ...
 - Diamètre d'un réseau de télécommunications (qualité de service)
 - Problèmes de Transport
 - Jeux (graphe dont les sommets sont les états du jeu et les arcs les transitions légales)
 - Investissements, ordonnancements
 - Navigation ...

2-1-Formulation Du Problème

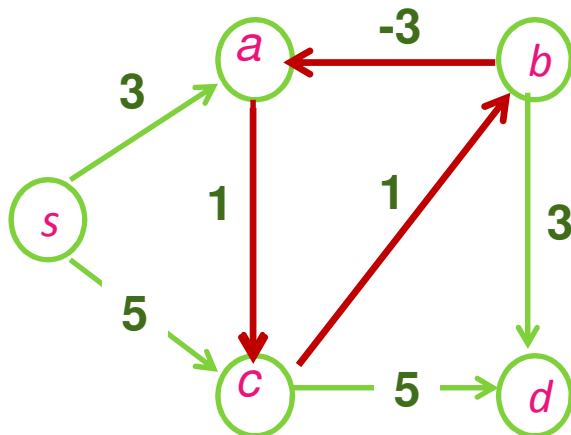
- Soit $G = (S, A, c)$ un graphe orienté et valué (réseau).
 - $c : A \rightarrow \mathbb{R}$
 - $c_u \equiv$ valeur, poids, ou coût de l'arc u .
- Le poids des arcs traduit un coût de parcours. Il peut s'agir d'une distance (réseau routier), d'un coût financier, d'une durée, d'un débit, etc...
- Le poids d'un chemin μ étant défini par: $c(\mu) = \sum_{u \in \mu} c_u$
- Le problème du chemin de valeur minimale d'un sommet x à un sommet y consiste à trouver un chemin μ^* de x vers y tel que :

$$c(\mu^*) = \min_{\mu} c(\mu) \quad \text{où } \mu \text{ chemin de } x \text{ à } y$$



Un PCC de s à d

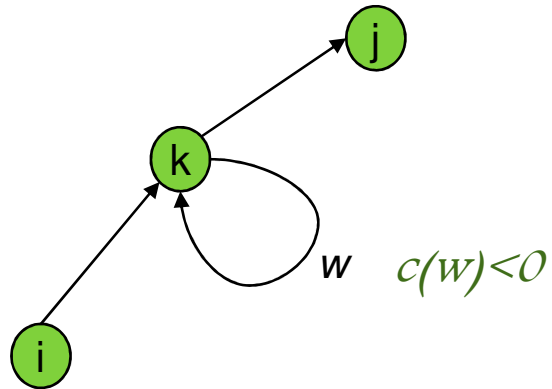
$\mu^*: s \rightarrow a \rightarrow c \rightarrow b \rightarrow d$; $c(\mu^*)=8$



Un PCC de s à d ? $-\infty$!

il n'en existe pas !!

Condition d'existence



- ❖ μ chemin non élémentaire de i à j avec un circuit w
- ❖ μ' restriction de μ n'empruntant pas w

- On a : $c(\mu) = c(\mu') + c(w)$
- Si $c(w) < 0$, il n'existe pas de chemin minimal de i à j .

Condition d'existence de chemins minimaux :

G ne doit pas contenir de circuits de coûts négatifs
(appelés : circuits absorbants)

- PROPRIÉTÉ: Existence d'un PCC

Il existe un PCC entre s et i ssi

- (a) i est atteignable depuis s (i.e. $\exists$ un chemin de s à i), et
- (b) il n'existe pas de circuit absorbant dans le graphe considéré

- PROPRIÉTÉ FONDAMENTALE DES PCC:

Si $\mu : s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_k$ est un PCC entre s_0 et s_k , alors tout sous-chemin $s_i \rightarrow \dots \rightarrow s_j$ (avec $0 \leq i < j \leq k$) de μ est un PCC de s_i à s_j .



Principe d'optimalité de Bellman

LES PROBLÈMES DE PCC

Les problèmes de plus courts chemins (P.C.C) se divisent en trois grands groupes :

- les PCC à origine unique : On cherche tous les PCC depuis un sommet de départ s ;
- les PCC à destination unique : On cherche tous les PCC menant à un sommet d'arrivée t ; et
- les PCC entre toutes les paires de sommets de G .



Algo. de Dijkstra et algo de Bellman = pour les PCC à origine unique.
Algo de Floyd = pour les PCC entre tous les sommets.

3-1-Les Algorithmes de recherche de plus court chemin

- Le cadre : les graphes orientés $G = (S, A, c)$ et valués avec $c: A \rightarrow \mathbb{R}$ (évidemment sans circuit négatif !)
- Le problème :
Etant donné un sommet s , trouver la distance d'un PCC entre s et tout autre sommet $i \in S$.
- Conventions :
 - $\pi^* (x)$: valeur d'un chemin minimal de s à x ,
 - $\pi (x) = +\infty$ s'il n'existe pas de chemin de s à x

Algorithme de Moore–Dijkstra

- $G(S,A,c)$: graphe orienté d'ordre n , valué par des coûts positifs.

$$c: A \rightarrow \mathbb{R}_+$$

- L'algorithme de M-D permet la résolution du problème du PCC d'un sommet (numéroté 1) à tous les autres. Il procédera en $(n-1)$ itérations.

- A une itération donnée, l'ensemble des sommets est partitionné en deux sous ensembles:

$$S_1 \text{ et } \bar{S}_1 = S \setminus S_1 \text{ avec } 1 \in S_1$$

- On calcule les plus courts chemins de proche en proche par ajustement successifs.

- On note par $\pi(i)$, la longueur d'un plus court chemin de 1 à i .

(11)

ALGORITHME DE MOORE-DIJKSTRA

1- Initialisation:

$$k := 0$$

$$\bar{S}_1 = \{2, 3, \dots, n\}, \quad \pi_k(1) = 0$$

$$\forall i \in \bar{S}_1 \quad \pi_k(i) = \begin{cases} c_{1i} & \text{si } (1, i) \in A \\ +\infty & \text{si non} \end{cases}$$

2- Sélectionner

$$j \in \bar{S}_1 / \pi_k(j) = \min_{i \in \bar{S}_1} \pi_k(i)$$

Faire $\bar{S}_1 := \bar{S}_1 \setminus \{j\}$ si $\bar{S}_1 = \emptyset \Rightarrow \text{Fin.}$
sinon aller à 3

3- Faire

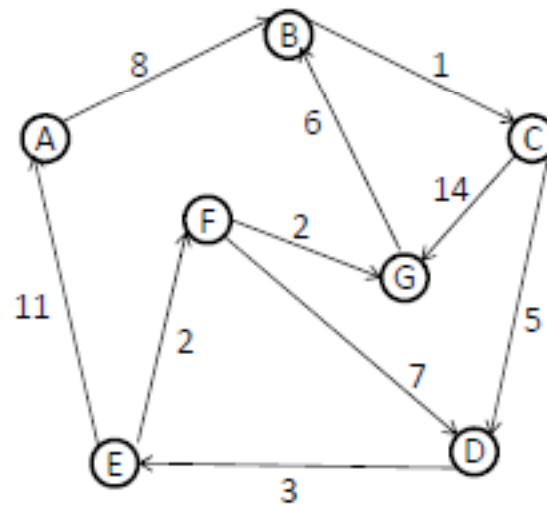
$$\forall i \in \bar{S}_1 / (j, i) \in A$$

$$\pi_{k+1}(i) = \min \left(\pi_k(i), \pi_k(j) + c_{ji} \right)$$

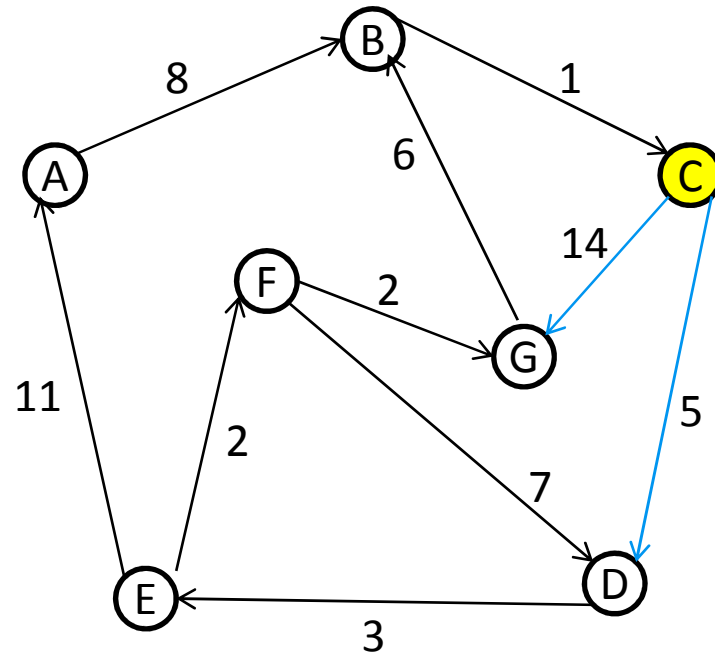
$k := k + 1$ et Retour à 2

Exemple-Dijkstra (détailé)

Recherche des plus courts chemins partant du sommet © vers tous les autres sommets, sur le graphe ci-dessous :

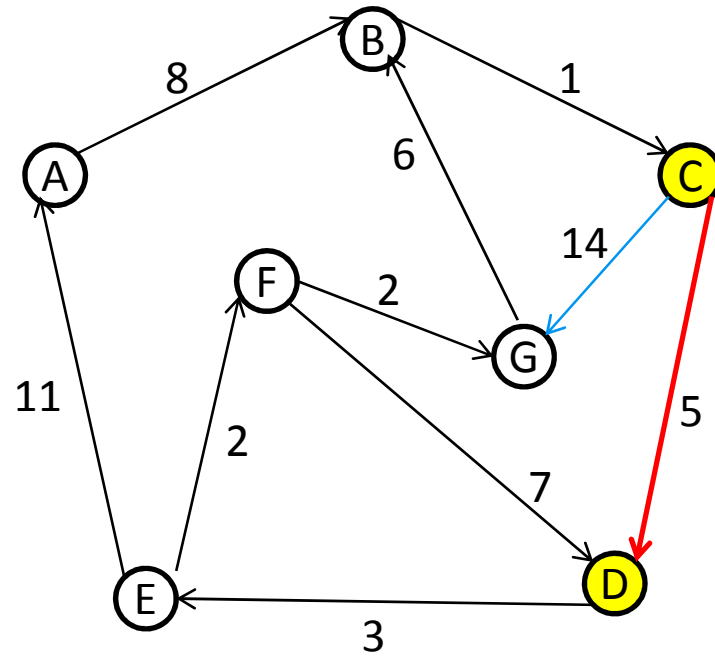


(13)

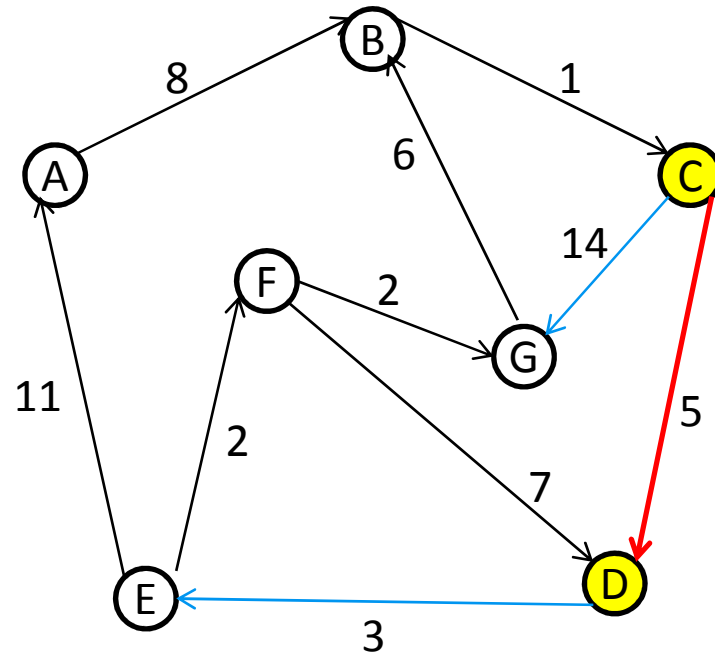


$\bar{S}_1 \backslash (S)$	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B,D,E,F,G}	0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14

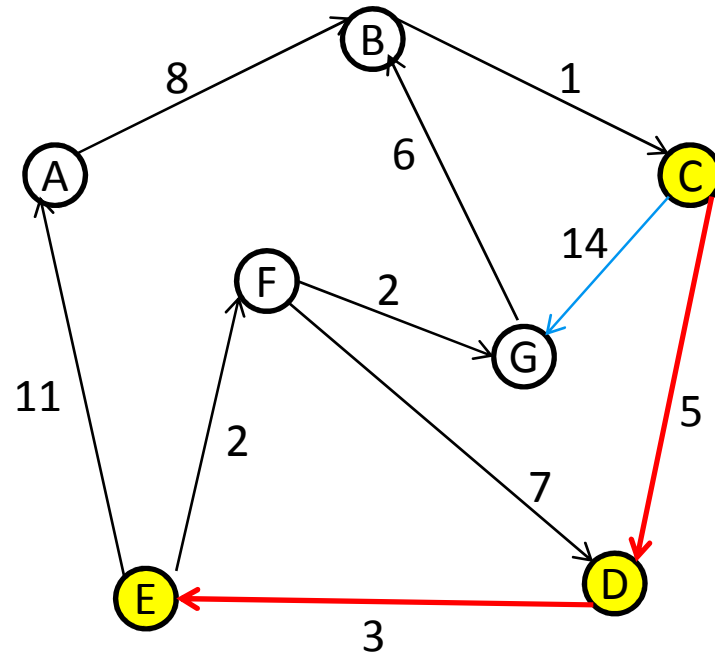
(14)



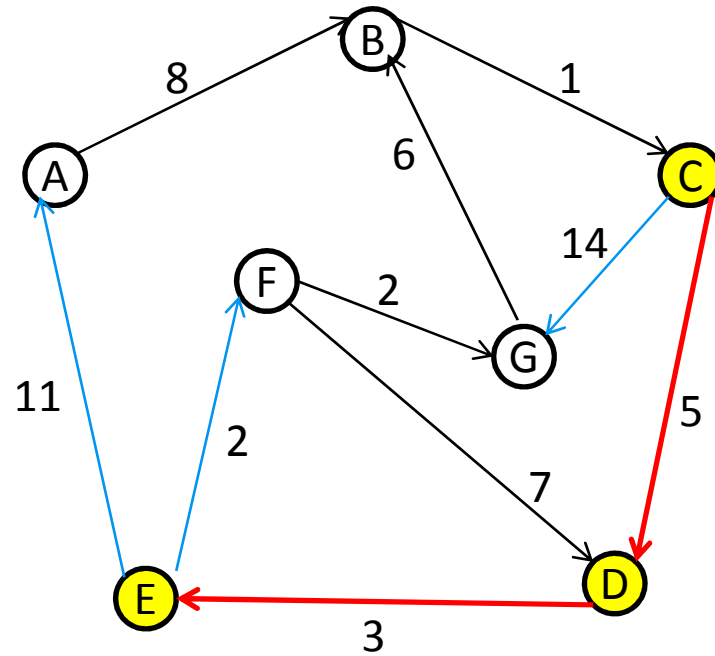
$\bar{S}_1 \backslash (S)$	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B, D ,E,F,G}	0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B,E,F,G}							



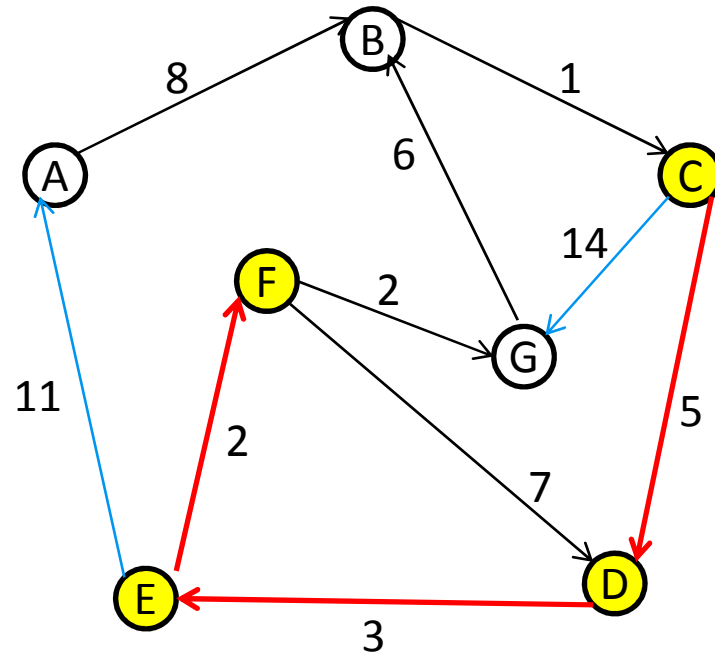
$\bar{S}_1 \backslash (s)$	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B,D,E,F,G}	0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B,E,F,G}		$+\infty$	$+\infty$		8	$+\infty$	14



$\bar{S}_1 \backslash (s)$	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B, D ,E,F,G}	0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B, E ,F,G}		$+\infty$	$+\infty$		8	$+\infty$	14
{A,B,F,G}							

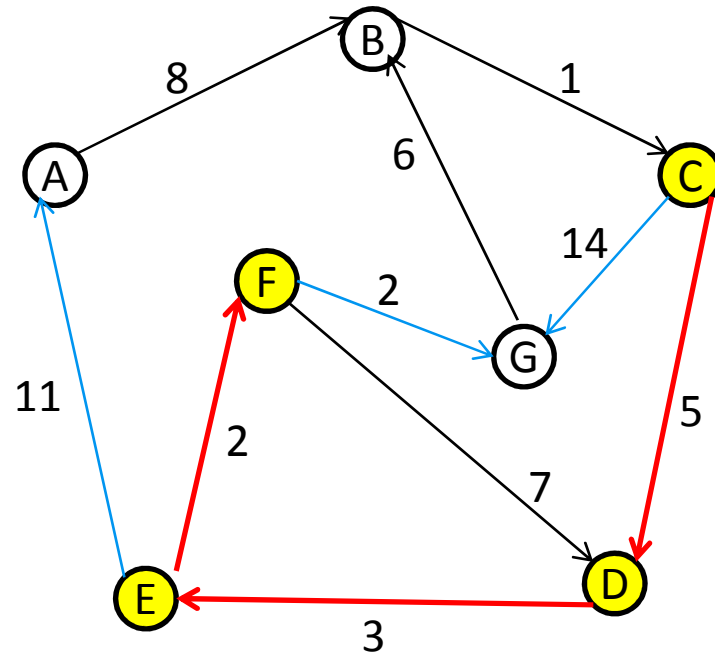


$\bar{S}_1 \backslash S$	(S)	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B,D,E,F,G}		0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B,E,F,G}			$+\infty$	$+\infty$		8	$+\infty$	14
{A,B,F,G}			19	∞			10	14

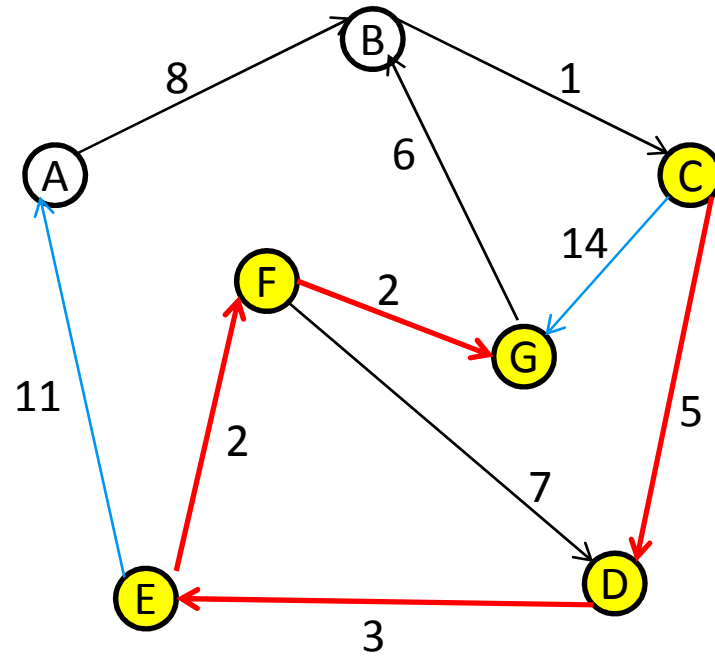


$\bar{S}_1 \backslash (S)$	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B,D,E,F,G}	0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B,E,F,G}		$+\infty$	$+\infty$		8	$+\infty$	14
{A,B,F,G}		19	$+\infty$			10	14
{A,B,G}							

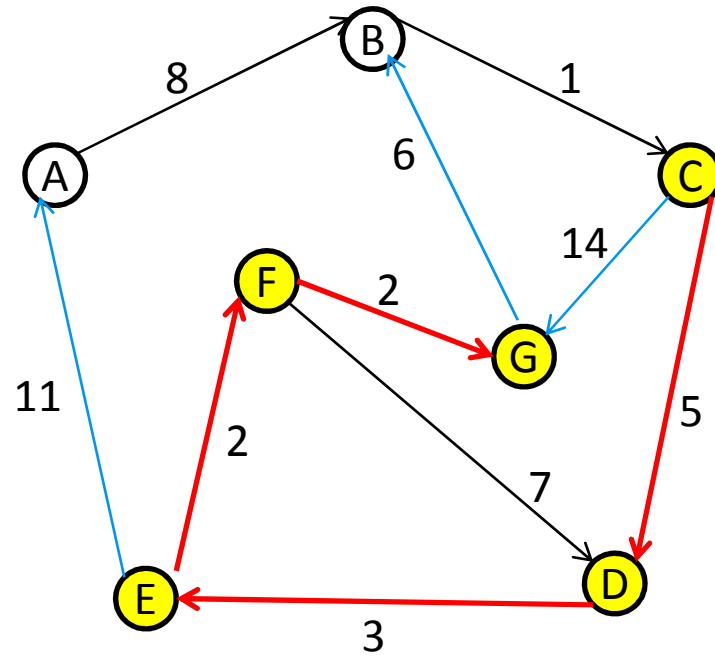
[19]



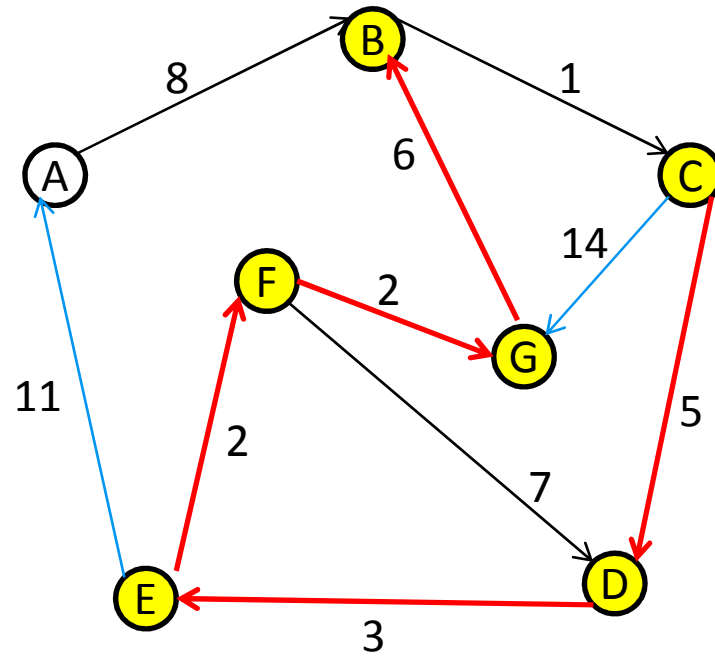
$\bar{S}_1 \backslash S$	(S)	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B,D,E,F,G}		0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B,E,F,G}			$+\infty$	$+\infty$		8	$+\infty$	14
{A,B,F,G}			19	$+\infty$			10	14
{A,B,G}			19	$+\infty$				12



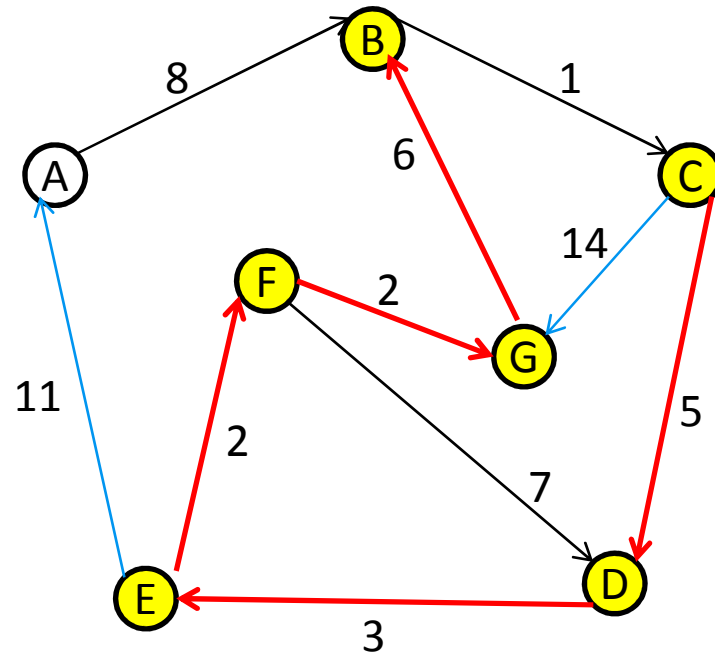
$\bar{S}_1 \backslash S$	(S)	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B,D,E,F,G}		0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B,E,F,G}			$+\infty$	$+\infty$		8	$+\infty$	14
{A,B,F,G}			19	$+\infty$			10	14
{A,B,G}			19	$+\infty$				12
{A,B}								



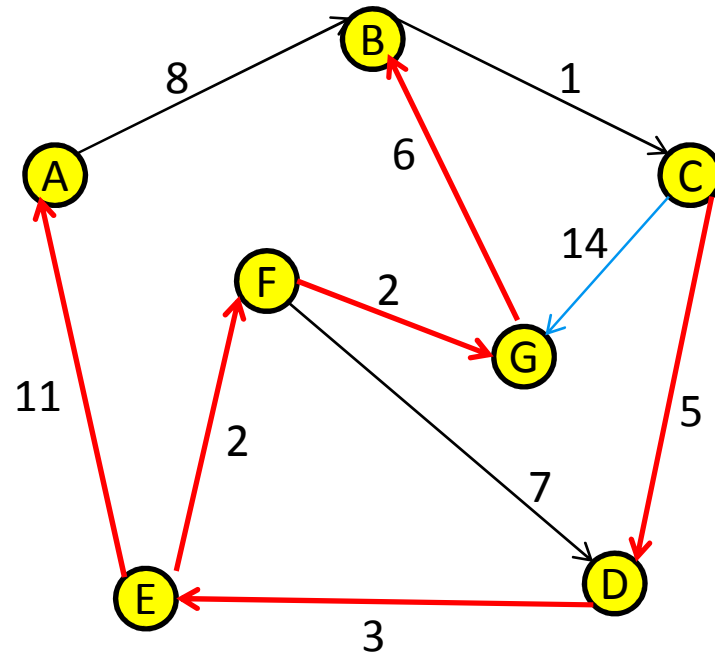
$\bar{S}_1 \backslash S$	(S)	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B,D,E,F,G}		0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B,E,F,G}			$+\infty$	$+\infty$		8	$+\infty$	14
{A,B,F,G}			19	$+\infty$			10	14
{A,B,G}			19	$+\infty$				12
{A,B}			19	18				



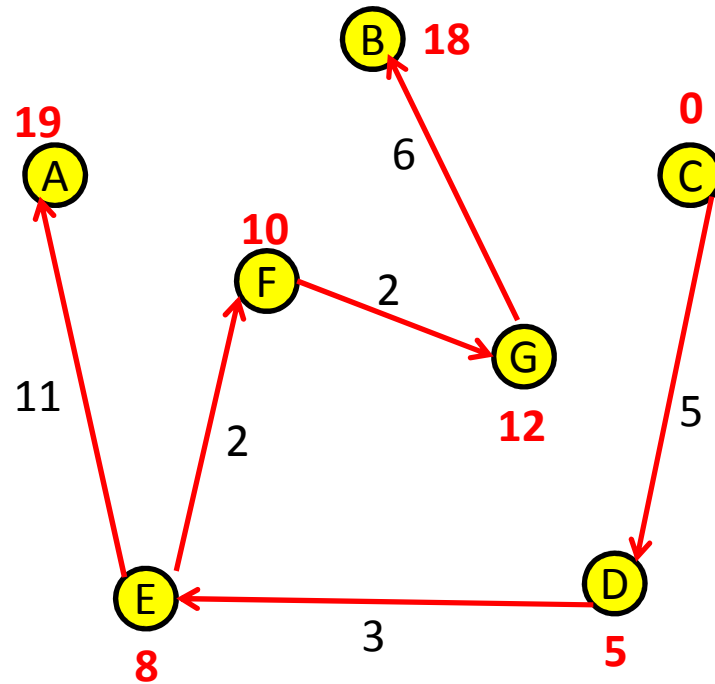
$\bar{S}_1 \backslash (s)$	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B,D,E,F,G}	0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B,E,F,G}		$+\infty$	$+\infty$		8	$+\infty$	14
{A,B,F,G}		19	$+\infty$			10	14
{A,B,G}		19	$+\infty$				12
{A,B}		19	18				
{A}							



$\bar{S}_1 \backslash (s)$	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B,D,E,F,G}	0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B,E,F,G}		$+\infty$	$+\infty$		8	$+\infty$	14
{A,B,F,G}		19	$+\infty$			10	14
{A,B,G}		19	$+\infty$				12
{A,B}		19	18				
{A}		19					



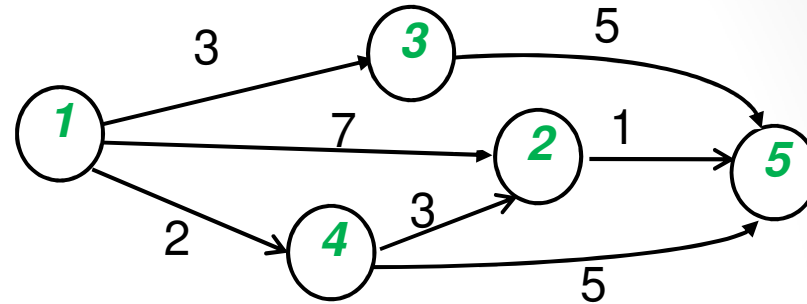
$\bar{S}_1 \backslash (S)$	(C)	(A)	(B)	(D)	(E)	(F)	(G)
{A,B,D,E,F,G}	0	$+\infty$	$+\infty$	5	$+\infty$	$+\infty$	14
{A,B,E,F,G}		$+\infty$	$+\infty$		8	$+\infty$	14
{A,B,F,G}		19	$+\infty$			10	14
{A,B,G}		19	$+\infty$				12
{A,B}		19	18				
{A}		19					
{ $\emptyset$ }	Fin						



L'arborescence des plus court chemins à partir de ③

⑤	③	①	②	④	⑥	⑦	⑧
$\pi^*(s)$	0	19	18	5	8	10	12

EXP2 : ALGORITHME DE DIJKSTRA



S_1	1	2	3	4	5
$\{1\}$	0	7	3	2	$+\infty$
$S_1 \cup \{4\}$	0	5_4	3	2	7_4
$S_1 \cup \{3\}$	0	5_4	3	2	7_4
$S_1 \cup \{2\}$	0	5_4	3	2	6_2
$S_1 \cup \{5\} = S$	0	5_4	3	2	6_2

Exp: P.C.C de 1 à 5

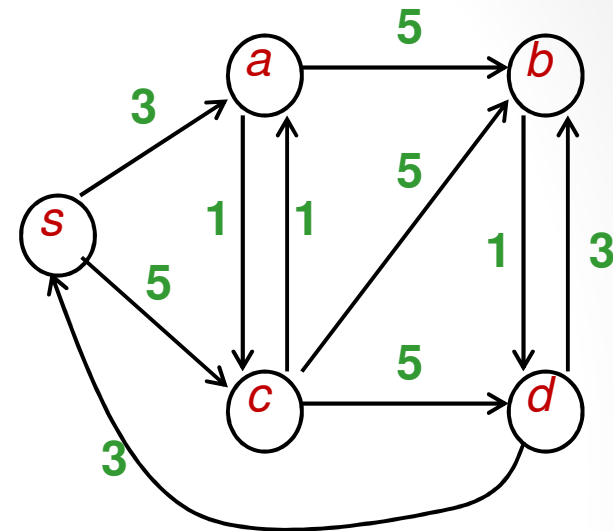
$\mu: 1 \rightarrow 4 \rightarrow 2 \rightarrow 5$

$\pi^*(s)$

[27]

EXP 3: ALGORITHME DE DIJKSTRA

S_1	s	a	b	c	d
$\{s\}$	0	3	$+\infty$	5	$+\infty$
$S_1 \cup \{a\}$	0	3	8_a	4_a	$+\infty$
$S_1 \cup \{c\}$	0	3	8_a	4_a	9_c
$S_1 \cup \{b\}$	0	3	8_a	4_a	$9_{c,b}$
$S_1 \cup \{d\} = S$	0	3	8_a	4_a	$9_{c,b}$



Exp: P.C.C de s à d , il en existe 2 :

- $\mu: s \rightarrow a \rightarrow b \rightarrow d$
- $\mu': s \rightarrow a \rightarrow c \rightarrow d$

$$c(\mu) = c(\mu') = 9$$

Algorithme de Bellman

- $G(S,A,c)$: graphe d'ordre n , valué par des longueurs de signes quelconques. $c: A \rightarrow \mathbb{R}$
- L'algorithme de Bellman permet la recherche du PCC d'un sommet I à tous les autres ou la détection d'un circuit absorbant dans un graphe valué par des longueurs de signes quelconques.
- A la $k^{\text{ème}}$ itération, il calcule la longueur du PCC de I à i contenant au plus k arcs. S'il n'existe pas de circuit absorbant, un chemin de I à i contiendrait au plus $(n-1)$ arcs.

ALGORITHME DE BELLMAN (VERSION NAÏVE)

1- Initialisation:

Posons $\pi_0(1) := 0$ et $\forall i \in S / \{1\} \quad \pi_0(i) := +\infty$

$k := 1$

2- Faire

$\pi_k(1) := 0,$

$\forall i \neq 1, \pi_k(i) := \min_{(j,i) \in U} (\pi_{k-1}(i), \pi_{k-1}(j) + c_{ji})$

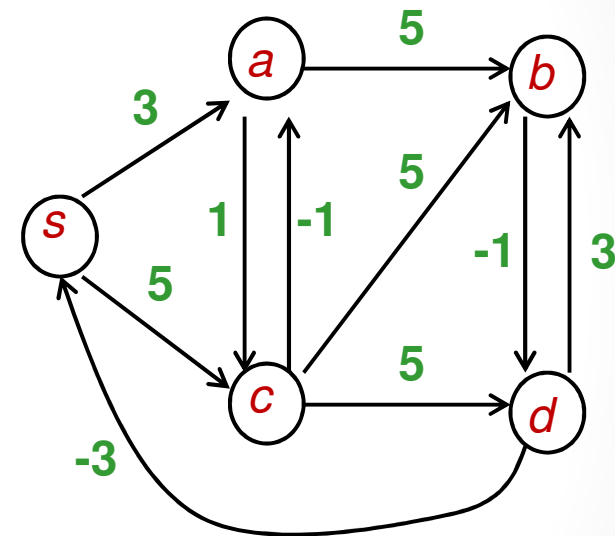
3- Si $\pi_k(i) = \pi_{k-1}(i) \quad \forall i \Rightarrow \text{Fin}$

Sinon $\begin{cases} \text{Si } k \leq n-1, k := k+1 \text{ et aller à 2} \\ \text{Si } k = n \Rightarrow \text{il existe un circuit absorbant} \end{cases}$

Exemples:

EXP 1 : ALGORITHME DE BELLMAN À partir du sommet s

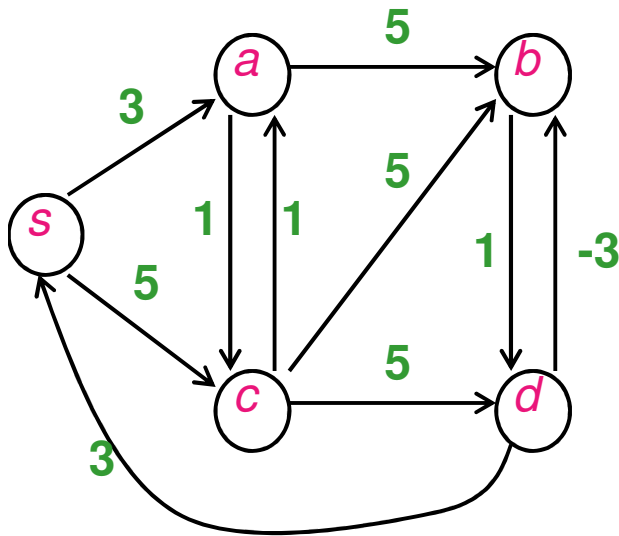
k	s	a	b	c	d
0	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	0	3_s	$+\infty$	5_s	$+\infty$
2	0	3_s	8_a	4_a	10_c
3	0	3_s	8_a	4_a	7_b
4	0	3_s	8_a	4_a	7_b



$$\pi_2(c) = \min(\underbrace{\pi_1(c)}_5, \underbrace{\pi_1(s) + c_{sc}}_5, \underbrace{\pi_1(a) + c_{ac}}_{3+1})$$

[31]

EXP 2 : ALGORITHME DE BELLMAN À partir du sommet (s)



k	s	a	b	c	d
0	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	0	3_s	$+\infty$	5_s	$+\infty$
2	0	3_s	8_a	4_a	10_c
3	0	3_s	7_d	4_a	9_b
4	0	3_s	6_d	4_a	8_b
5	0	3_s	5_d	4_a	7_b

$\exists i \in S \text{ tq } \pi_k(i) \neq \pi_{k-1}(i) \text{ et } k = n$



Détection d'un circuit absorbant

[32]

Rang d'un sommet

REMARQUE :

L'algorithme de Bellman converge rapidement si les sommets sont visité par ordre de **rang** croissant .

- Dans un graphe **sans circuit** il existe toujours un sommet n'ayant aucun précédent. On appelle 1 ce sommet .

Définition :

On défini le rang d'un sommet dans un graphe sans circuit par :

$$\text{rg}(1) = 0$$

$\text{rg}(i)$ = le nombre d'arcs dans un chemin de 1 à i
de cardinalité (en nombre d'arcs) maximale.

Algorithme de Rang

Soit $G(S,A)$ un graphe **sans circuit**

1- Initialisation: $S, k:=0$

2- $S_k := \{\text{sommets sans précédents dans } S\}$

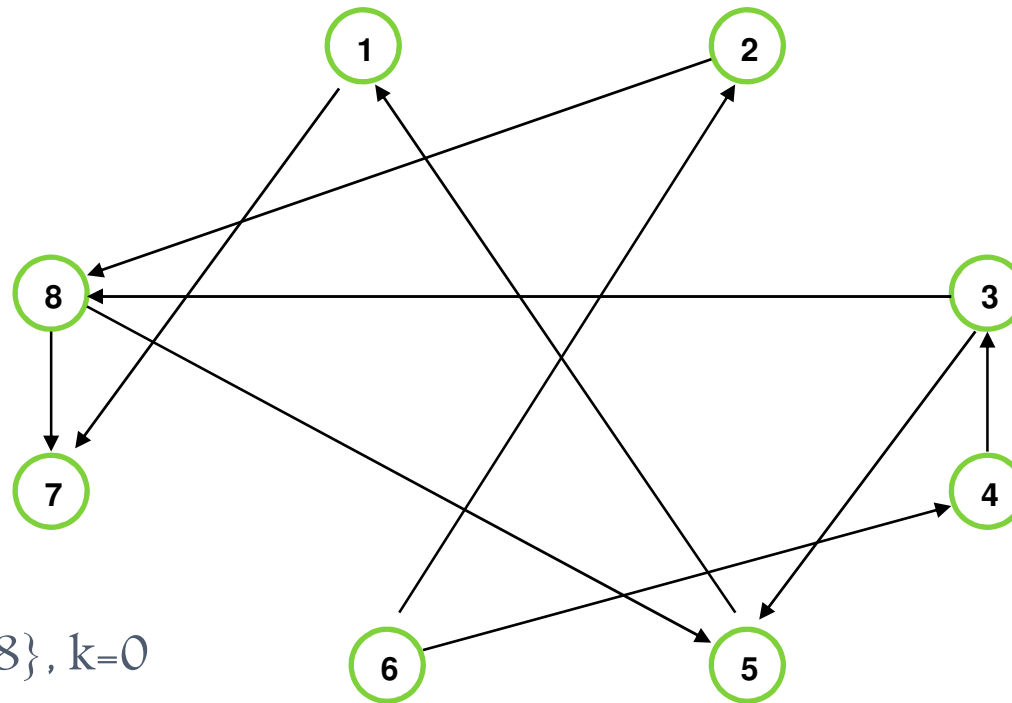
$\forall i \in S_k, \text{rg}(i) = k$

$S := S \setminus S_k$

3- Si $S = \emptyset$ FIN

Sinon $k := k+1$ et retour à 2

EXAMPLE:



$S = \{1, 2, 3, 4, 5, 6, 7, 8\}, k=0$

$S_0 = \{6\} \rightarrow r(6) = 0 ; S = \{1, 2, 3, 4, 5, 7, 8\}, k=1$

$S_1 = \{2, 4\} \rightarrow r(2) = r(4) = 1 ; S = \{1, 3, 5, 7, 8\}, k=2$

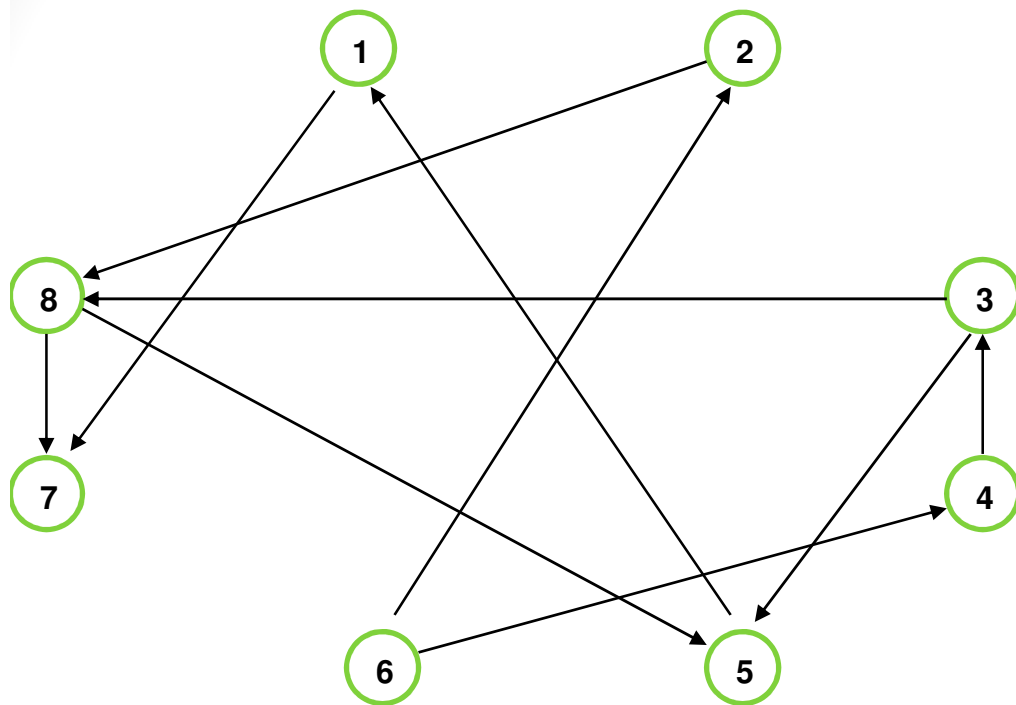
$S_2 = \{3\} \rightarrow r(3) = 2 ; S = \{1, 5, 7, 8\}, k=3$

$S_3 = \{8\} \rightarrow r(8) = 3 ; S = \{1, 5, 7\}, k=4$

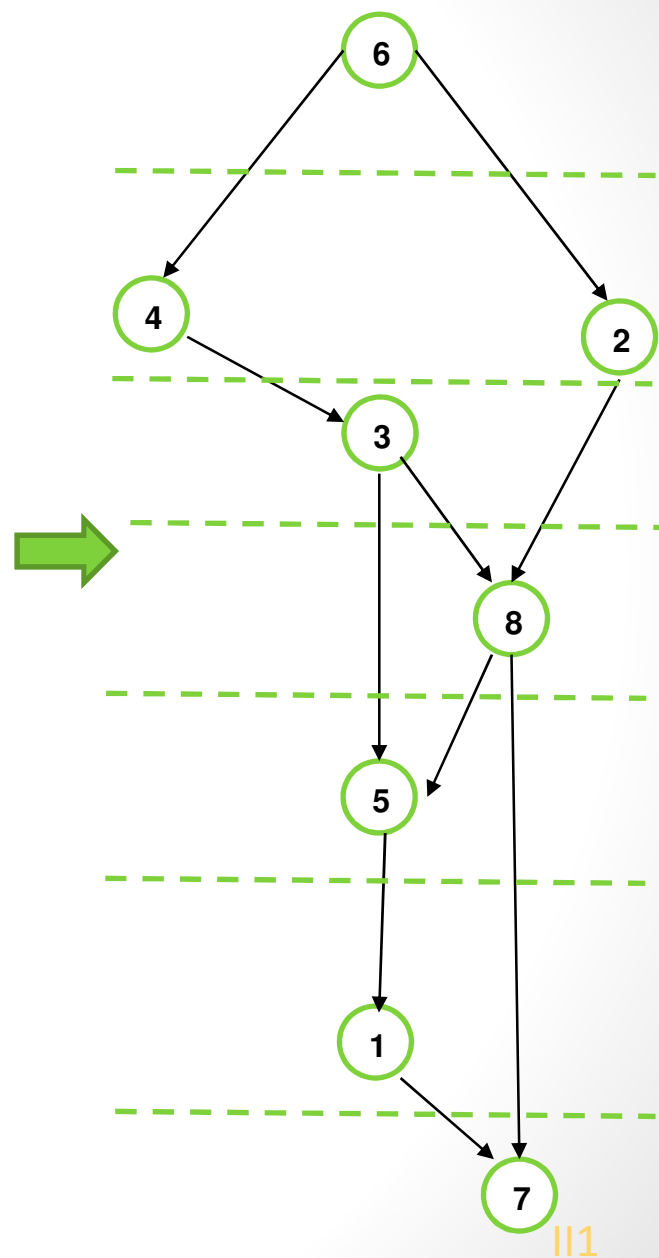
$S_4 = \{5\} \rightarrow r(5) = 4 ; S = \{1, 7\}, k=5$

$S_5 = \{1\} \rightarrow r(1) = 5 ; S = \{7\}, k=6$

$S_6 = \{7\} \rightarrow r(7) = 6 ; S = \emptyset$



N.CHAOUACHI

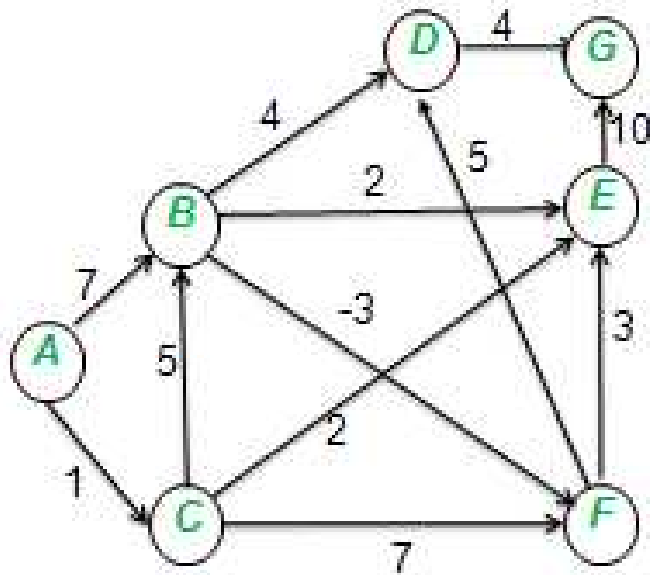


PCC

[36]

||1

EXP 3: Recherche du P.C.C de A à G.



$$k=0, S_0=\{A\}, r(A)=0$$

$$S_1=\{C\}, r(C)=1$$

$$S_2=\{B\}, r(B)=2$$

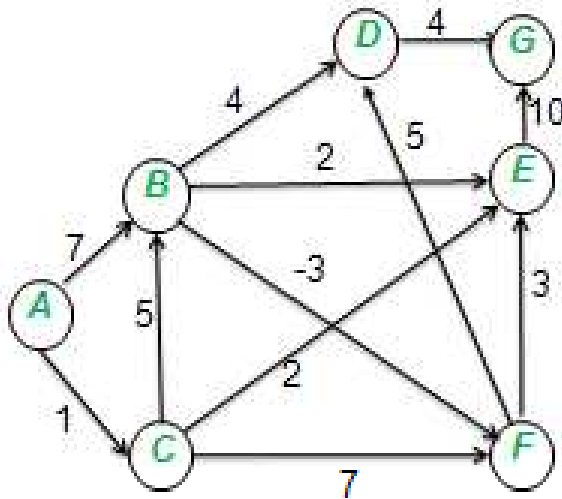
$$S_3=\{F\}, r(F)=3$$

$$S_4=\{D, E\}, r(D)=r(E)=4$$

$$S_5=\{G\}, r(G)=5$$

$$S_6=\emptyset$$

Recherche du P.C.C de A à G après avoir ranger les sommets par rang croissant :



Algorithme de Bellman:

k	A	C	B	F	D	E	G
0	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	0	1_A	7_A	$+\infty$	$+\infty$	$+\infty$	$+\infty$
2	0	1_A	6_C	4_B	11_B	3_C	$+\infty$
3	0	1_A	6_C	3_B	9_F	3_C	13_E
4	0	1_A	6_C	3_B	8_F	3_C	12_D
5	0	1_A	6_C	3_B	8_F	3_C	12_D

Pcc de A à G :

$A \rightarrow C \rightarrow B \rightarrow F \rightarrow D \rightarrow G$

de longueur 12

ALGORITHME DE BELLMAN AMÉLIORÉ

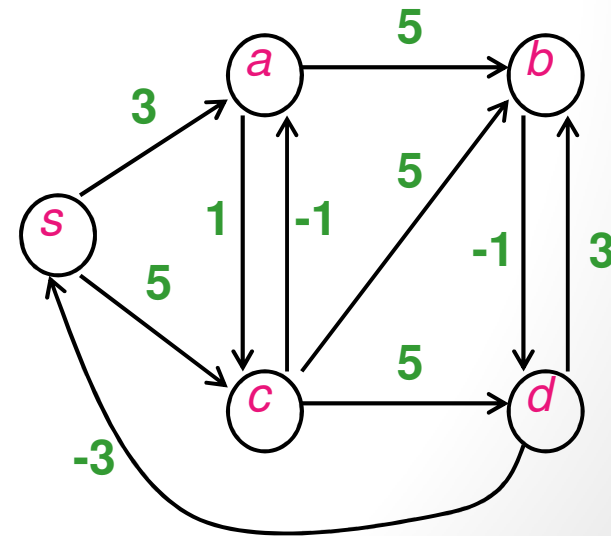
- Une amélioration de l'algorithme de Bellman peut se faire en utilisant les résultats de l'itération en cours , i.e. : l'instruction 2 devient:

$$\pi_k(1) := 0,$$

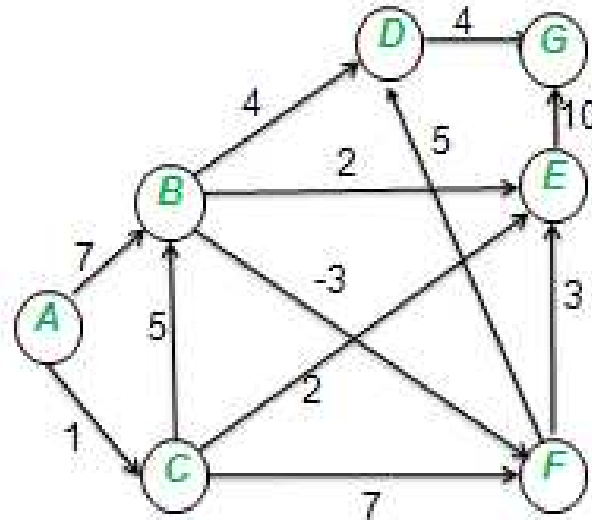
$$\forall i \neq 1, \pi_k(i) := \min_{(j,i) \in U} (\pi_{k-1}(i), \pi_{k-1}(j) + c_{ji}, \pi_k(j) + c_{ji})$$

- Retour à l'Exp1 (slide 31) avec l'alg de Bellman amélioré:

k	s	a	b	c	d
0	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	0	3_s	8_a	4_a	7_b
2	0	3_s	8_a	4_a	7_b



Retour à l'Exp3 (slide 37):



Algorithme de Bellman amélioré:

k	A	C	B	F	D	E	G
0	0	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$	$+\infty$
1	0	1_A	6_C	3_B	8_F	3_C	12_D
2	0	1_A	6_C	3_B	8_F	3_C	12_D

Pcc de A à G :

A $\longrightarrow$ C $\longrightarrow$ B $\longrightarrow$ F $\longrightarrow$ D $\longrightarrow$ G

de longueur 12

Partie 2:

PROBLÈMES DÉRIVÉS:

PCC ENTRE N'IMPORTE QUELLE

PAIRE DE SOMMETS

1-2-ALGORITHME DE FLOYD-WARSHALL

- $G(S,A,c)$: graphe d'ordre n , valué par des longueurs de signes quelconques. ($c : A \rightarrow \mathbb{R}$)
- L'algorithme de Floyd-Warshall permet la recherche du pcc entre toute paire de sommets ou la détection d'un circuit absorbant dans un graphe valué par des longueurs de signes quelconques.
- On définit les matrices:

$$L = (l_{ij})_{1 \leq i, j \leq n}$$

$$\text{où } l_{ij} = \begin{cases} l_{ij} & \text{si } (i,j) \in U \\ 0 & \text{si } i = j \\ +\infty & \text{sinon} \end{cases}$$

$$L^* = (l^*_{ij})_{1 \leq i, j \leq n}$$

$$\text{où } l^*_{ij} = \begin{cases} c_{\mu^*} & \text{où } \mu^* \text{ est le p.c.c de } i \text{ à } j \\ +\infty & \text{si un tel chemin } \nexists \end{cases}$$

- L'algorithme de Floyd-Warshall permet de déterminer L^* à partir de L en exactement n itération et ce en passant par les matrices $L^{(k)}$ ainsi définies :

$$L^{(k)} = (l_{ij}^k)_{1 \leq i, j \leq n}$$

$$L^{(0)} = (l_{ij}^0)_{1 \leq i, j \leq n} = (l_{ij})_{1 \leq i, j \leq n}$$

$$\forall k \geq 1 \quad l_{ij}^k = \min \{ l_{ij}^{k-1}, l_{ik}^{k-1} + l_{kj}^{k-1} \}$$

- l_{ij}^1 = longueur du p.c.c de i à j ne pouvant avoir que 1 comme sommet intermédiaire
- l_{ij}^k = longueur du p.c.c de i à j ne pouvant avoir que des sommets intermédiaires appartenant à $\{1, 2, \dots, k\}$

- Pour pouvoir lire le p.c.c , il faut tenir compte des sommets intermédiaires. Pour cela on introduit une seconde matrice P définie par

$$P^{(0)} = (p_{ij}^{(0)})_{1 \leq i, j \leq n}$$

$$\text{où } p_{ij}^{(0)} = \begin{cases} 0 & \text{si } l_{ij} = +\infty \\ i & \text{sinon} \end{cases}$$

- A chaque itération k , la matrice P est mise à jour :

$$\text{si } l_{ij}^k = l_{ik}^{k-1} + l_{kj}^{k-1} \quad \text{alors } p_{ij}^k = p_{kj}^{k-1}$$

ALGORITHME DE FLOYD-WARSHALL

pour $k=1$ à n

pour $i=1$ à n sauf k

si $l_{ik} + l_{ki} < 0$ FIN % présence d'un circuit négatif

si $l_{ik} \neq \infty$ alors

pour $j=1$ à n sauf i

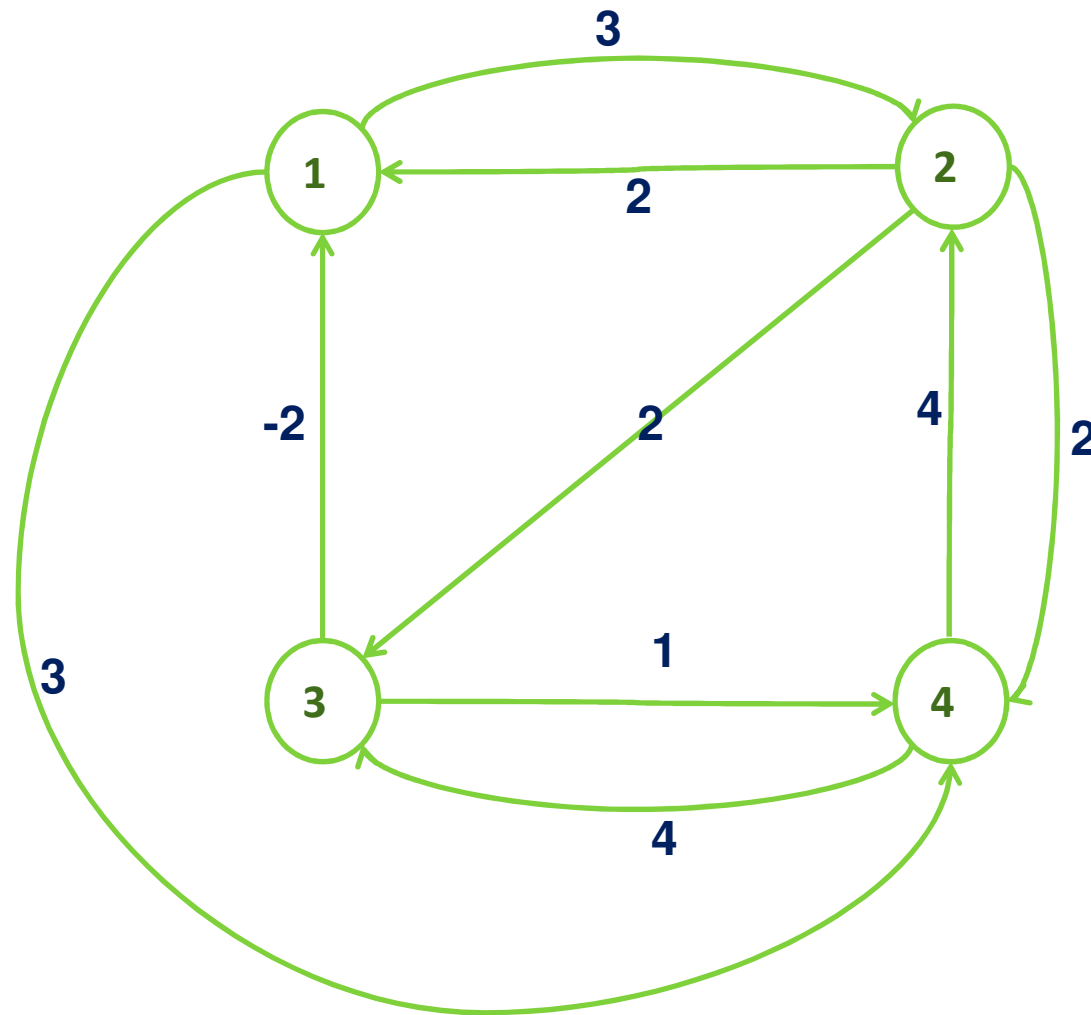
si $l_{ik} + l_{kj} < l_{ij}$

$l_{ij} \leftarrow l_{ik} + l_{kj}$

$p_{ij} \leftarrow p_{kj}$

Complexité : $O(n^3)$.

Exemple.



$$L^{(0)} = \begin{pmatrix} 0 & 3 & \infty & 3 \\ 2 & 0 & 2 & 2 \\ -2 & \infty & 0 & 1 \\ \infty & 4 & 4 & 0 \end{pmatrix}$$

$$P^{(0)} = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 0 & 3 & 3 \\ 0 & 4 & 4 & 4 \end{pmatrix}$$

$$L^{(1)} = \begin{pmatrix} 0 & 3 & \infty & 3 \\ 2 & 0 & 2 & 2 \\ -2 & \mathbf{1} & 0 & 1 \\ \infty & 4 & 4 & 0 \end{pmatrix}$$

$$P^{(1)} = \begin{pmatrix} 1 & 1 & 0 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & \mathbf{1} & 3 & 3 \\ 0 & 4 & 4 & 4 \end{pmatrix}$$

$$L^{(2)} = \begin{pmatrix} 0 & 3 & \mathbf{5} & 3 \\ 2 & 0 & 2 & 2 \\ -2 & 1 & 0 & 1 \\ \mathbf{6} & 4 & 4 & 0 \end{pmatrix}$$

$$P^{(2)} = \begin{pmatrix} 1 & 1 & \mathbf{2} & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 1 & 3 & 3 \\ \mathbf{2} & 4 & 4 & 4 \end{pmatrix}$$

$$L^{(3)} = \begin{pmatrix} 0 & 3 & 5 & 3 \\ \mathbf{0} & 0 & 2 & 2 \\ -2 & 1 & 0 & 1 \\ \mathbf{2} & 4 & 4 & 0 \end{pmatrix}$$

$$P^{(3)} = \begin{pmatrix} 1 & 1 & 2 & 1 \\ \mathbf{3} & 2 & 2 & 2 \\ 3 & 1 & 3 & 3 \\ \mathbf{3} & 4 & 4 & 4 \end{pmatrix}$$

$$L^{(4)} = \begin{pmatrix} 0 & 3 & 5 & 3 \\ 0 & 0 & 2 & 2 \\ -2 & 1 & 0 & 1 \\ 2 & 4 & 4 & 0 \end{pmatrix} = L^*$$

$$P^{(4)} = \begin{pmatrix} 1 & 1 & 2 & 1 \\ 3 & 2 & 2 & 2 \\ 3 & 1 & 3 & 3 \\ 3 & 4 & 4 & 4 \end{pmatrix} = P^*$$

OBTENTION DES PLUS COURT CHEMIN:

Pour obtenir un plus court chemin de i à j , il suffit d'utiliser la $i^{\text{ème}}$ ligne de la dernière matrice P^* .

Par exemple, si on veut obtenir le plus court chemin μ du sommet 4 au sommet 1, on consulte la matrice $P^{(4)}$ ainsi :

$P_{41} = 3 \Rightarrow 3$ est donc le prédécesseur de 1 dans μ

$P_{43} = 4 \Rightarrow 4$ est donc le prédécesseur de 3 dans ce même chemin μ

Finalement, $\mu : 4 \rightarrow 3 \rightarrow 1$

Partie 3 : Application du P.C.C

Le problème central de l'Ordonnancement (PERT-MPM)

(50)

1-3-INTRODUCTION:

La réalisation de projets complexes comme par exemple la construction d'un barrage, l'installation d'une nouvelle chaîne de production, l'installation d'un nouveau système d'information,... requiert *une planification au préalable et un contrôle au cours de l'exécution*. Les problèmes liés à ces objectifs sont des problèmes d'ordonnancement



On cherche un *ordre d'exécution* des différentes tâches de manière à optimiser un certain critère



CALENDRIER D'EXÉCUTION

- Résoudre un problème d'ordonnancement = trouver l'ordre et le calendrier suivant lequel devront être exécutées les tâches afin d'optimiser un projet donné en tenant compte d'un certain nombre de contraintes
- Les contraintes sont de type:
 - Potentiel : contraintes d'antériorité (ou postériorité),
contrainte de localisation temporelle
 - Disjonctif: La non réalisation simultanée de 2 tâches
 - Cumulatif : les moyens disponibles (hommes, machines, budgets...)

2-3-Le problème central de l'ordonnancement

- Les seules contraintes considérées dans ce problème sont les contraintes d'antériorité
- On cherche à réaliser un ensemble de tâches appelé projet, chaque tâche est caractérisée par sa durée et par les contraintes qui la lie aux autres tâches
- **Objectif :** Finir le projet le plus tôt possible
- La représentation de ce problème par un graphe permet d'identifier les tâches prioritaire et donne pour chaque tâche le temps d'exécution permettant de finir le projet au plus tôt

Le graphe potentiel-tâches (M.P.M)

Méthode des Potentiels METRA.

À partir d'un prj donné on construit le graphe suivant.

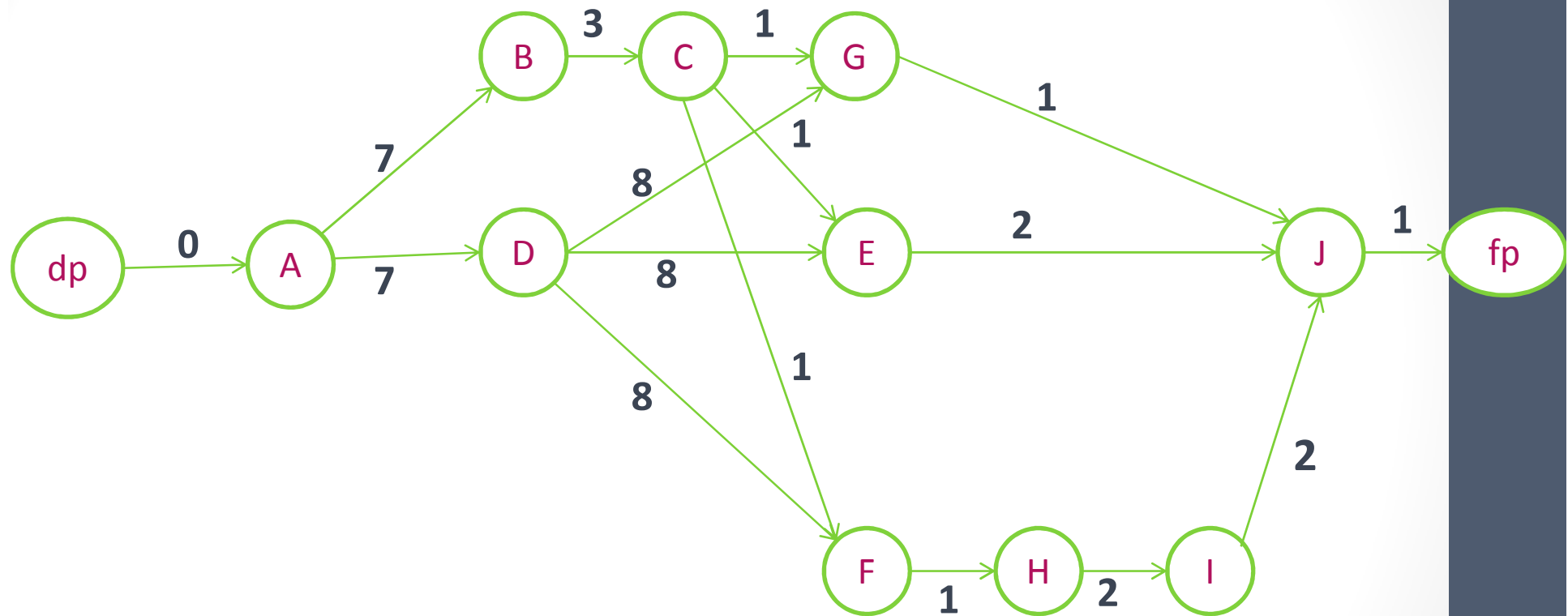
- ▶ À chaque tâche i on associe un sommet du graphe
- ▶ On définit l'arc (i,j) si la tâche i précède la tâche j
- ▶ c_{ij} représente la durée d'exécution de la tâche i
- ▶ On rajoute deux sommets fictifs :
 - dp correspondant au début du prj et reliés aux sommet sans précédents
 - fp correspondant à la fin du prj et relié aux sommet sans suivants

Le graphe ainsi construit doit être sans circuit

Exemple 1.3: Construction d'une maison

Étapes	Temps requis	Préalables
A- Gros-oeuvre	7	-
B-Charpente de la toiture	3	A
C- Toiture	1	B
D- Installation sanitaire et électrique	8	A
E-Façade	2	D,C
F-Fenêtres	1	D,C
G- Aménagement du jardin	1	D,C
H- Travaux de plafonnage	2	F
I- Peinture	2	H
J- Emménagement	1	E,G,I

Le graphe potentiel-tâches de l'exemple 1.3 (M.P.M)



Question: quelle est la durée **minimale** de réalisation de ce projet?

[56]

Exemple 2.3: Tracer le graphe potentiels-Tâches associé.

Tâches	A	B	C	D	E	F	G	H	I
Durée d'exécution	1	2	5	4	3	1	2	1	3
Tâches antérieures	---	A	A	A	B	C	C,E	D	B,H

Pour chaque tâche, on définit la date du début d'exécution au plus tôt et au plus tard permettant de finir le prj au plus tôt

Algorithmes:

1-Calcul des dates au plus tôt

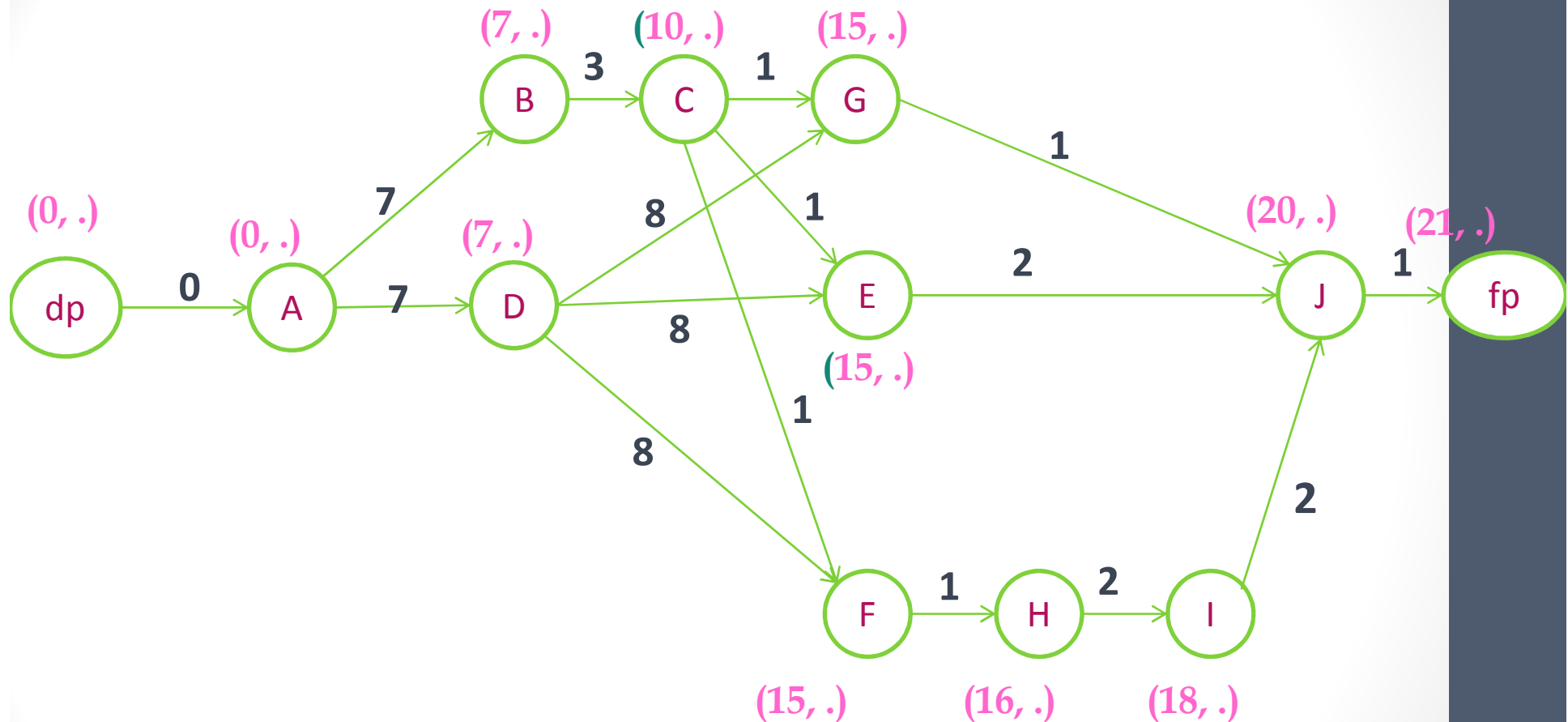
1. Prendre les sommet par rang ↗

2. Faire $t_{dp}=0$;

3. $t_j = \max (t_i + d_i) / i \in \Gamma^-(j)$

- t_j représente la longueur du plus long chemin de dp à j
- t_{fp} est le tps min pour réaliser le projet = la longueur du plus long chemin de dp à fp

Retour à l'exemple 1.3



Réponse: la durée **minimale** de réalisation de ce projet est de 21 u.t

2-Calcul des dates au plus tard

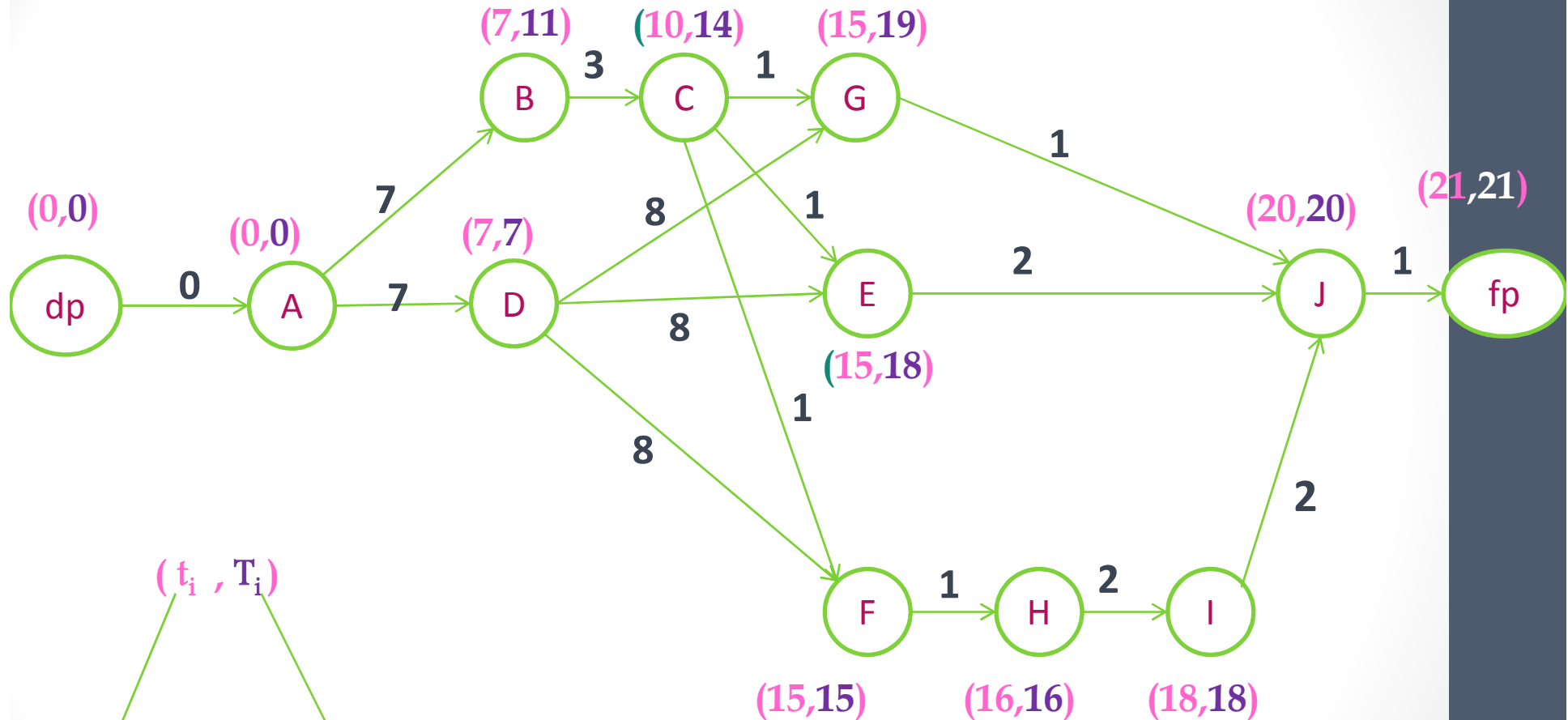
1. Prendre les sommet par rang $\searrow$
2. $T_{fp}=t_f$;
3. $T_j= \min (T_i- d_j) / i \in \Gamma^+(j)$

1. On appelle marge m_i de la tâche i , $m_i = T_i - t_i$
2. Les tâches dont les marges sont nulles sont dites tâches critiques
3. On appelle chemin critique, tout chemin de **dp** à **fp** passant par les sommets critiques et de longueur égale à la durée min du projet.

Remarques:

- La contrainte i précède j , s'écrit $t_j \geq t_i + d_i$, c'est-à-dire $t_j - t_i \geq d_i$
- Si un retard est pris sur une des tâches critiques, la durée minimale du projet sera augmentée d'autant.

Retour à l'exemple 1.3

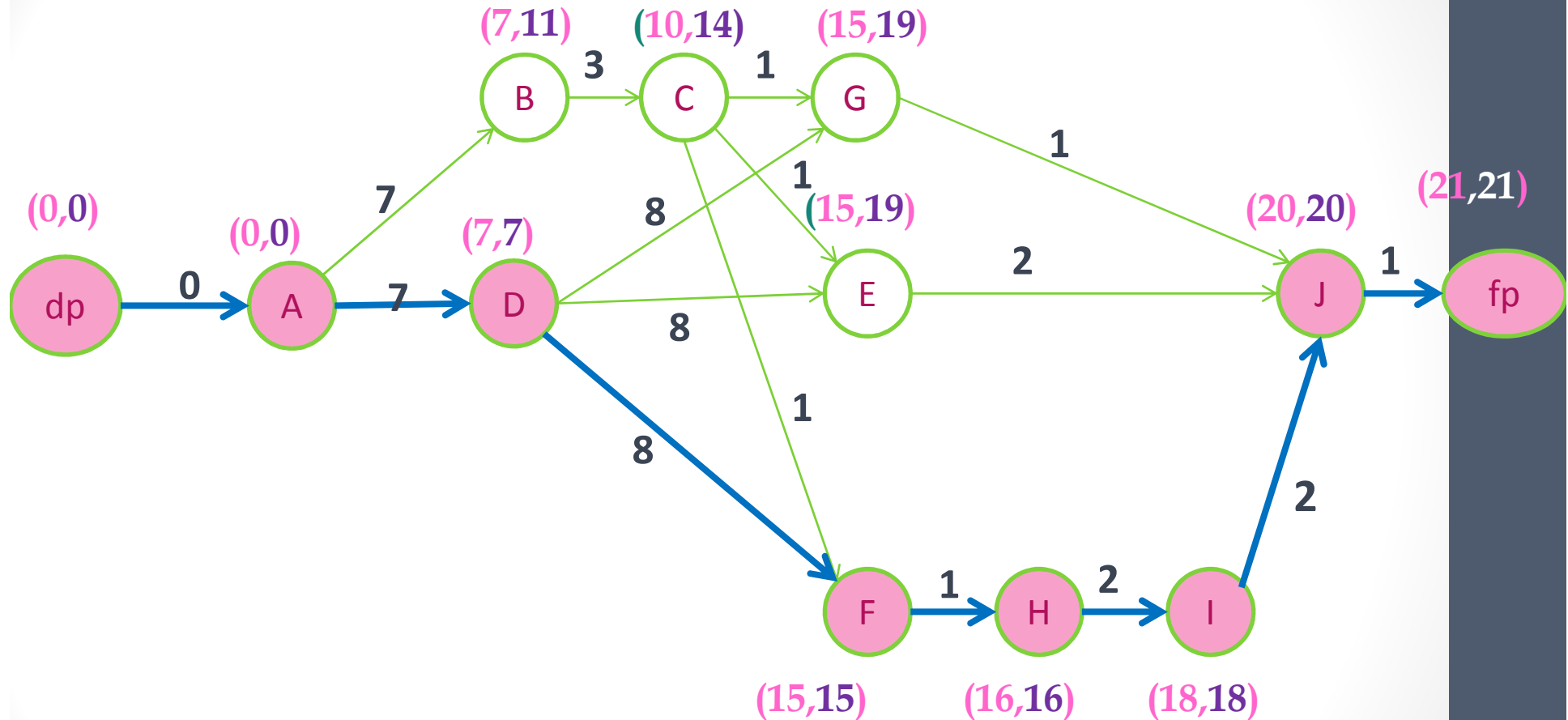


(t_i, T_i)

dates au
plus tôt

dates au
plus tard

Chemin critique pour l'exemple 1.3 :

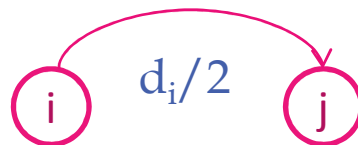


[63]

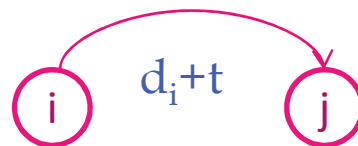
TRADUCTION DES CONTRAINTES

Si les contraintes de type potentiel ne sont pas uniquement des contraintes d'antériorité, on modifie le graphe potentiel-tâches ainsi:

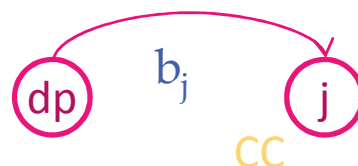
- ◆ La contrainte j ne doit pas commencer avant la moitié du tps de réalisation de la tâche i se représente par un arc (i,j) de valeur $d_i/2$



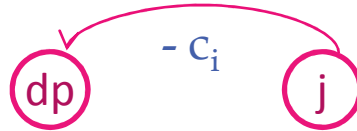
- ◆ La contrainte j ne doit commencer qu'après un temps t de la fin de i se traduit par un arc(i,j) de valeur $d_i + t$



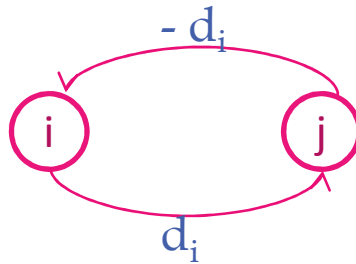
- ◆ La contrainte j ne doit commencer qu'après la date b_j se représente par un arc(dp,j) de durée b_j



- ◆ La contrainte j doit commencer avant la date c_j se traduit par un arc entre dp et j de durée $-c_j$



- ◆ La contrainte j doit suivre immédiatement la tâche i se représente par un circuit entre (i,j)



REMARQUES

Attention, l'ajout des contraintes peut introduire des circuits dans le graphe:

- 1) Circuit à coût négatif : longueur du P.L.C = durée min des travaux
- 2) Circuit à coût positif (absorbant) : pas de solution

Graphe potentiel-étapes (P.E.R.T)

Program Evaluation and Review Technic

À partir d'un prj donné on construit le graphe suivant:

- ▶ Chaque tâche est un arc de longueur d_i (durée de i)
- ▶ Les sommets $\rightarrow$ Étapes du prj , i.e. le début et la fin de chaque tâche
- ▶ Si une tâche j succède à une tâche i , l'extrémité initiale de j coïncide avec l'extrémité terminale de i
- ▶ On rajoute au graphe deux étapes fictives : dp et fp

$\Rightarrow$ On définit ainsi un graphe sans circuit .

Règles de construction :

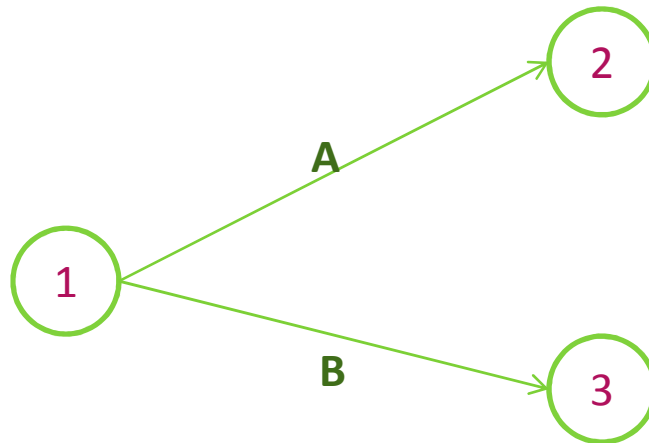
1. Toute tâche a une étape de début et une étape de fin.
Une tâche suivante ne peut démarrer que si la tâche précédente est terminée.



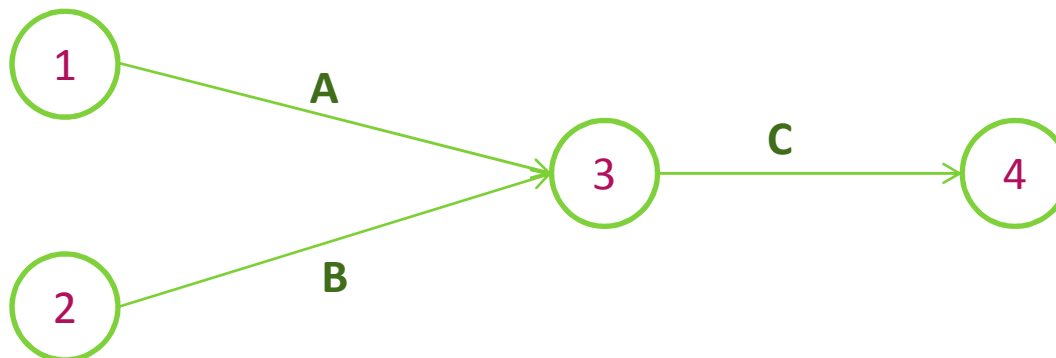
2. Deux tâches qui se succèdent immédiatement sont représentées par des flèches qui se suivent.



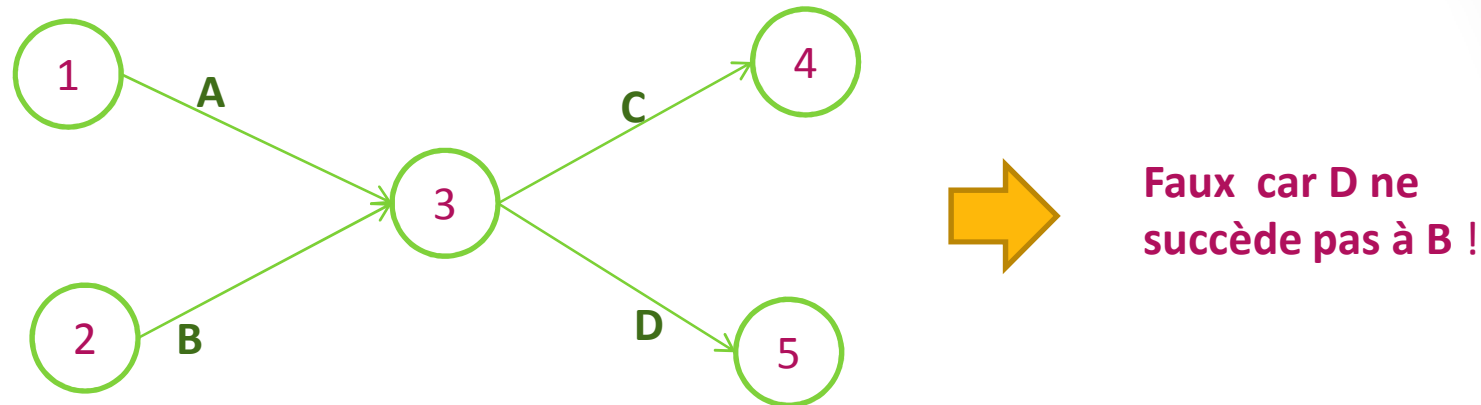
3. Deux tâches A et B qui sont simultanées sont représentées de la manière suivante :



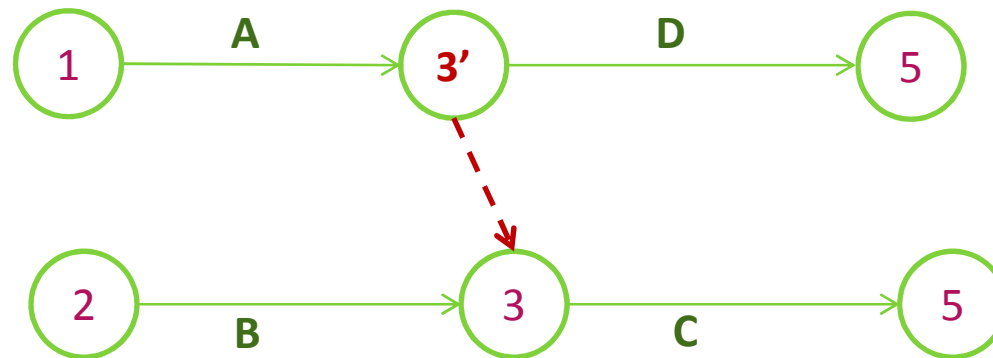
4. Deux tâches A et B qui sont convergentes (c'est à dire qui précèdent une même tâche C) sont représentées de la manière suivante



Par exemple, si la tâche C succède aux tâches A et B, et que la tâche D succède seulement à la tâche A.



5. on représente le problème de la manière suivante : on introduit une **tâche fictive** de durée nulle (Elle ne modifie pas le délai final) :



- ▶ Les dates au plus tôt et au plus tard de l'exécution d'une tâche sont calculées de la même manière que pour le graphe potentiel-tâche.
- ▶ La durée min du projet est la longueur du plus long chemin entre le début et la fin du projet.

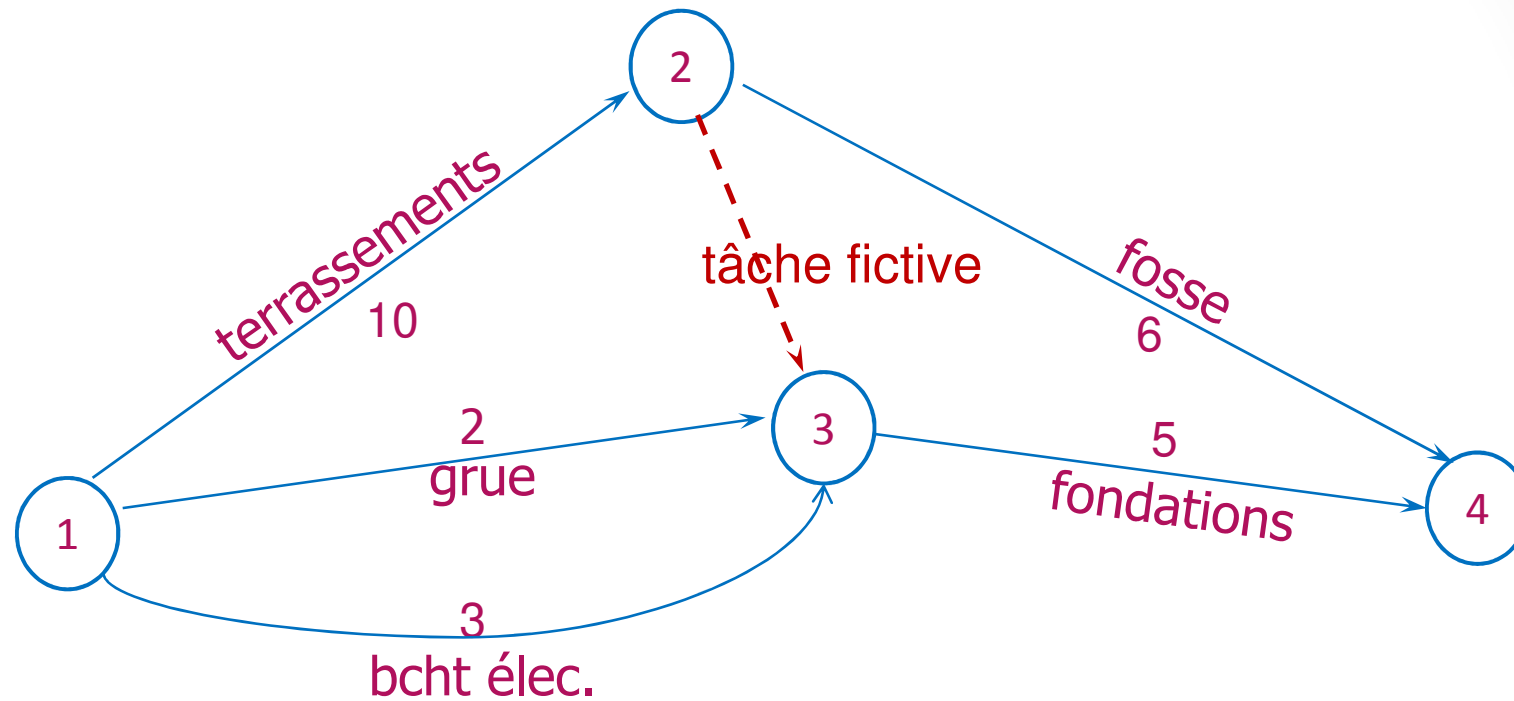
Exemple 3.3:

Code de la tâche	Description	Durée (en jours)
A	Exécution des terrassements	10
B	Mise en place de la grue	2
C	Fondations	5
D	Branchement électrique	3
E	Installation de la fosse septique	6

Contraintes

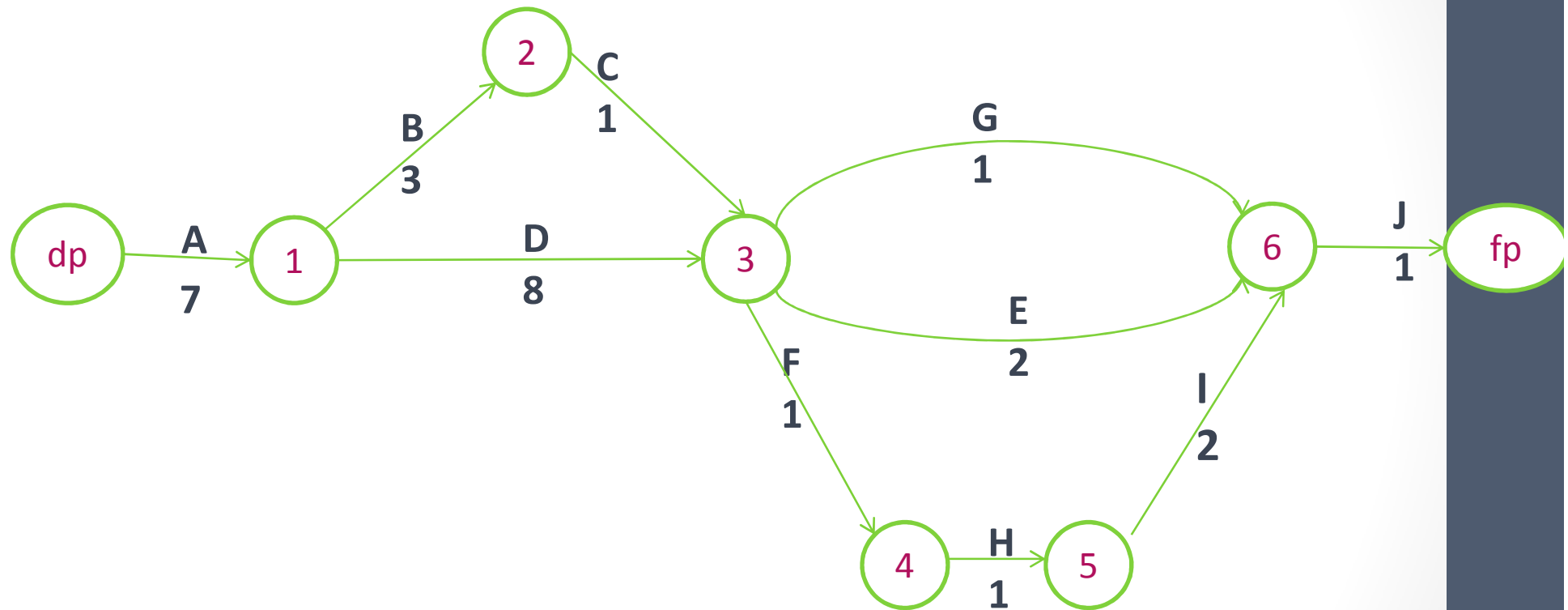
- ☞ On a besoin du branchement électrique pour passer aux fondations.
- ☞ On a besoin de la grue pour les fondations.
- ☞ L'installation de la fosse septique et les fondations ne peuvent être exécutés que si les travaux de terrassement sont terminés.

Le graphe potentiel-étapes de l'exemple 3.3 (P.E.R.T)



Remarque: On aura un **multigraphe**

Le graphe potentiel-étapes de l'exemple 1.3 (P.E.R.T)



[73]

LE DIAGRAMME DE GANTT

1. Principe:

On représente au sein d'un tableau, en ligne les différentes tâches et en colonne les unités de temps(exprimées en mois, semaines, jours, heures...) La durée d'exécution d'une tâche est matérialisée par un trait au sein du diagramme.

2. Réalisation:

1. On détermine les différentes tâches (ou opérations) à réaliser et leur durée.
2. on définit les relations d'antériorité entre tâches.
3. on représente d'abord les tâches n' ayant aucune antériorité, puis les tâches dont les tâches antérieures ont déjà été représentées, et ainsi de suite...
4. on représente par un trait parallèle en pointillé à la tâche planifiée la progression réelle du travail.

Exemple 4.3: Tracer le graphe potentiels-Tâches associé au projet suivant, ainsi que le diagramme de Gantt

Tâche	Durée	Tâches antérieures
A	2	-
B	4	-
C	4	A
D	5	A, B
E	6	C,D

Le graphe potentiel-tâches (M.P.M) de l'exemple 4.3

